Faculty of Engineering and Technology

Electrical and Computer Engineering Department

ENCS5343

Second Project

Text-to-Video Retrieval using Deep Learning

Prepared by:

Sadeen Ryahi – 1211109
Donia Al-Shiakh– 1210517

Instructor: Dr.Aziz Qaroush

Sec: 1

Date: February 4, 2026

Abstract

Text-to-video retrieval aims to retrieve relevant videos from a large database given a natural language query. This project presents a dual-encoder deep learning system that aligns video and text representations into a shared embedding space using contrastive learning. The system is evaluated on the MSVD benchmark and demonstrates competitive retrieval performance while remaining computationally efficient.

Contents

Abstract **2**

1 Introduction **5**
1.1 Problem Definition 5
1.2 Why This Problem Is Challenging 5
1.3 Project Goal and Approach 5

2 System Architecture and Design Choices **6**
2.1 Overall Architecture: Why Dual-Encoder? 6
2.2 Video Encoder: Processing Visual Information 6
2.3 Text Encoder: Processing Language 7
2.4 Similarity and Retrieval in the Shared Space 7
2.5 Key Design Decisions and Justifications 8

3 Dataset and Experimental Setup **8**
3.1 MSVD Dataset: Description and Choice 8
3.2 Data Preprocessing Pipeline 9
3.3 Training Setup 9
3.4 Training Hyperparameters 10
3.5 Evaluation Metrics Explained 10

4 Results and Discussion **10**
4.1 Final Results 10
4.2 Detailed Analysis of Results 11
4.3 Why Recall@1 Is Much Lower Than Recall@5 and Recall@10 11
4.4 Failure Cases and Limitations 11
4.5 What Works Well 12
4.6 Retrieval Efficiency 12

5 Qualitative Results **12**

6 Comparative Analysis **14**
6.1 Comparison with Fusion-Based Methods 14
6.2 Comparison with Large Pretrained Models (CLIP) 14
6.3 Comparison with Older Non-Deep-Learning Methods 15
6.4 Quantitative Comparison on MSVD 15

7 Conclusion and Future Work **16**
7.1 Project Summary 16
7.2 Strengths of the System 16

7.3 Limitations of the System 16
7.4 Future Directions 16
7.5 Concluding Remarks 17

1 Introduction

1.1 Problem Definition

Text-to-video retrieval is the task of finding relevant videos from a database in response to a natural language text query. Given a user's written description of what they want to see, the system returns a ranked list of videos where the most similar videos appear first. For example, if a user searches for "a person playing tennis," the system should return videos of people playing tennis at the top of the results list, with videos of other sports or activities appearing lower.

1.2 Why This Problem Is Challenging

Solving text-to-video retrieval involves several connected challenges. First, videos and text are fundamentally different types of information. A video contains visual and temporal information—it shows objects, actions, colors, and movements across time. Text contains linguistic information—semantic meaning expressed through words and grammar. Second, videos contain both static visual content (objects in frames) and dynamic temporal content (how things move and change). Describing all this meaning with just a few sentences of text means a lot of information is missing or implied. Third, there is a conceptual gap between human language and visual representation. Different people might describe the same video differently, and the same description might apply to multiple different videos. A retrieval system must learn to bridge this gap and find common ground between these very different modalities.

Additionally, videos can be quite long, and extracting useful features from all frames would be computationally expensive. Text descriptions can vary greatly in length and complexity. The system must handle this variability while still producing meaningful comparisons between any video-text pair.

1.3 Project Goal and Approach

The goal of this project is to build a practical text-to-video retrieval system that can efficiently search through a collection of videos to find those most relevant to a given text query. We use transfer learning to leverage knowledge from pretrained models. The core idea is a dual-encoder architecture: one neural network processes videos independently to produce video representations, and another processes text to produce text representations. Both representations exist in the same space, allowing direct comparison. We train this system using contrastive learning, which teaches the model to place matching video-text pairs close together in this shared space while pushing non-matching pairs apart. This approach balances perfor-

mance with practicality the system trains in a reasonable time and runs efficiently at test time.

2 System Architecture and Design Choices

2.1 Overall Architecture: Why Dual-Encoder?

Our system uses a dual-encoder architecture rather than other possible designs. In a dual-encoder setup, videos and text are processed by separate neural networks that do not interact directly during feature extraction. Only at the end do we compare their outputs.

We chose a dual-encoder design for practical reasons:

- **Efficiency at test time:** video embeddings can be precomputed and stored. When a user submits a query, we only encode the text once and compare it against all stored video embeddings.
- **Flexibility:** the video and text encoders can be reused independently, and the design can be extended to video-to-video retrieval.
- **Specialization:** each encoder can focus on learning representations suited to its modality without early cross-modal mixing.

2.2 Video Encoder: Processing Visual Information

The video encoder converts a video into a fixed-size vector representation in three stages.

Stage 1: Frame Sampling and CNN Feature Extraction

Processing every frame is expensive and redundant, so we uniformly sample 4 frames from each video, spaced equally across the duration. Each sampled frame is resized to the standard input resolution for the visual backbone. The sampled frames are processed by a pretrained CNN (ResNet-18), originally trained on ImageNet. Using a lightweight backbone reduces computational cost while maintaining competitive performance on the MSVD dataset. We uniformly sample 4 frames from each video, spaced equally across the video duration. This choice reflects a trade-off between temporal coverage and computational efficiency under GPU constraints.

Stage 2: Temporal Aggregation

After the CNN processes all frames, we have one feature vector per frame. We combine them into a single video-level representation using average pooling (taking the mean across the 4 frame features). This method is simple and fast and works well for short videos like those in MSVD. However, it assigns equal importance to all frames and does not explicitly model temporal order.

Stage 3: Projection to Shared Embedding Space

The pooled video feature vector is then transformed into a 256-dimensional video embedding using a projection layer. This projection adapts the visual features to the retrieval task. Finally, we apply L2 normalization so that similarity comparisons depend on vector direction rather than magnitude.

2.3 Text Encoder: Processing Language

The text encoder converts a sentence into a 256-dimensional embedding in three stages.

Stage 1: Tokenization

Text is converted into token IDs using the DistilBERT tokenizer. Tokenization splits text into subword units to handle rare words effectively. Captions are padded or truncated to a fixed length of 32 tokens to support efficient batching.

Stage 2: Transformer Language Encoding

Token IDs are processed by DistilBERT, a pretrained Transformer model. It produces contextual representations that capture word meaning in context. A single sentence representation is obtained by using the sequence-level pooled output (commonly based on the special classification token representation).

Stage 3: Projection to Shared Embedding Space

The sentence representation is passed through a projection layer to map it into the same 256-dimensional embedding space as the video embeddings. L2 normalization is applied so both modalities are comparable using the same similarity measure.

2.4 Similarity and Retrieval in the Shared Space

After encoding, each video and text query is represented by a normalized 256-dimensional vector. We compare text and video embeddings using cosine similarity.

During training, we learn representations that increase similarity for matching video-text pairs and decrease similarity for mismatched pairs using contrastive learning. At test time, retrieval is performed by encoding the query once and ranking all videos by similarity to the query embedding.

2.5 Key Design Decisions and Justifications

We made several choices that shaped the system:

- **Pretrained models (transfer learning):** using pretrained ResNet-18 and DistilBERT reduces training time and improves performance on limited data.
- **Four frames per video:** 4 frames balance content coverage and computational cost for short video clips.
- **Average pooling:** simple temporal aggregation is efficient and stable, though it may miss detailed temporal ordering.
- **256-dimensional embedding space:** a practical trade-off between memory/compute and representation capacity.
- **Cosine similarity:** appropriate for normalized embeddings and fast to compute.

3 Dataset and Experimental Setup

3.1 MSVD Dataset: Description and Choice

The Microsoft Video Description (MSVD) dataset is a standard benchmark for video understanding tasks. It contains approximately 1,970 short YouTube video clips and multiple English descriptions per video. Videos typically last 10–30 seconds and cover everyday activities such as cooking, sports, and animals.

We chose MSVD because it is widely used in research, enabling meaningful comparison to prior work. It is also small enough to train within a semester on modest hardware while still being large enough to evaluate a deep learning approach. We follow the standard train/validation/test split used in prior work to ensure fair evaluation and to avoid overfitting.

3.2 Data Preprocessing Pipeline

Video Preprocessing

We extract 8 frames per video, from which 4 frames are used during training to reduce computational cost. We apply standard ImageNet normalization to match the pretrained ResNet expectations. For efficiency, frames (or extracted features) are cached to disk so training does not repeatedly decode videos.

Text Preprocessing

Captions are cleaned using lightweight processing (lowercasing and trimming whitespace). We tokenize using DistilBERT's tokenizer, pad shorter descriptions, and truncate longer descriptions to 32 tokens for batching.

Storage and Caching

To speed up training, preprocessed data is cached. Video frames/features and tokenized captions are stored to reduce repeated preprocessing overhead.

3.3 Training Setup

Framework: PyTorch
Hardware: NVIDIA GPU (as available)
Backbones: ResNet-18 (video), DistilBERT (text)

We initialize both encoders with pretrained weights and allow fine-tuning during training. In each iteration, we encode a batch of paired videos and captions. We compute similarities between each text embedding and all video embeddings in the batch. Contrastive learning encourages matching pairs to rank higher than mismatched pairs.

3.4 Training Hyperparameters

Table 1 summarizes the main training hyperparameters used in our experiments.

Table 1: Training configuration.

Parameter	Value
Video backbone	ResNet-18 (pretrained)
Text encoder	DistilBERT (pretrained)
Frames per video	4 (uniform sampling)
Max text length	32 tokens
Embedding dimension	256
Batch size	32
Optimizer	AdamW
Learning rate	1×10^{-4}
Epochs	5
Temperature (contrastive)	0.07

3.5 Evaluation Metrics Explained

We evaluate retrieval performance using standard metrics:

- **Recall@K:** percentage of queries where the correct video appears in the top K results (K = 1, 5, 10).
- **Mean Rank:** average rank position of the correct video across queries (lower is better).
- **Median Rank:** median rank position across queries (less sensitive to outliers than mean rank).
- **mAP:** rewards ranking the correct video higher and summarizes ranking quality across queries.

4 Results and Discussion

4.1 Final Results

After training for 5 epochs, we evaluated the model on the MSVD test split. Table 2 summarizes the final results.

Table 2: Text-to-video retrieval results on MSVD test split

Metric	**Value**
Recall@1	20.27%
Recall@5	48.86%
Recall@10	61.58%
Mean Rank	17.22
Median Rank	6
mAP	0.3391

4.2 Detailed Analysis of Results

Recall@1 of 20.27% means that for about one-fifth of the queries, the correct video is ranked first. Recall@5 of 48.86% indicates that the correct video appears within the top 5 results for nearly half of the queries. Recall@10 of 61.58% suggests that in most cases, the model places the correct video within the top 10 retrieved results, though there is still a significant gap to perfect retrieval.

The Mean Rank of 17.22 indicates that, on average, the correct video appears around position 17, which shows that while many predictions are reasonably good, there are also numerous difficult cases where the correct match is ranked much lower. The Median Rank of 6 being substantially lower than the Mean Rank suggests a skewed distribution: many queries are retrieved fairly well, but some hard outliers significantly increase the average rank. The mAP value of 0.3391 reflects moderate retrieval quality and highlights that the model captures some cross-modal alignment but still struggles with fine-grained matching.

4.3 Why Recall@1 Is Much Lower Than Recall@5 and Recall@10

Recall@1 is the strictest metric because the model must select the single best match with no margin for error. Many text queries in MSVD are ambiguous and can correspond to multiple visually similar videos (e.g., "someone cooking food"). In such cases, the correct video is often ranked within the top few results but not necessarily first. This explains the large gap between Recall@1 (20.27%) and Recall@5 (48.86%). The higher Recall@5 and Recall@10 values indicate that the model generally places relevant videos near the top, even if the exact ranking order is not optimal.

4.4 Failure Cases and Limitations

Qualitative analysis revealed several common failure modes:

- **Ambiguous descriptions:** vague captions may match multiple videos, leading to ranking confusion.
- **Rare activities:** actions with limited training examples are not well represented in the learned embedding space.
- **Fine-grained distinctions:** visually similar actions (e.g., running vs. jogging) are difficult to separate.
- **Temporal dynamics:** using only a small number of frames with simple aggregation may fail to capture motion patterns.
- **Language variability:** uncommon wording or noisy captions can reduce text–video alignment.

4.5 What Works Well

Despite the limitations, the model performs relatively better on:

- **Clear and frequent activities:** common actions and objects seen often during training are retrieved more reliably.
- **Distinct visual content:** captions describing unique objects or scenes are easier to match.
- **Simple descriptions:** short and direct captions align better with the learned representations.

4.6 Retrieval Efficiency

A practical advantage of this system is fast inference. Since video embeddings can be precomputed offline, query-time retrieval only requires encoding the input sentence once and computing similarity scores against stored video embeddings. This makes the system computationally efficient and suitable for scalable retrieval settings, even though retrieval accuracy still has room for improvement.

5 Qualitative Results

To further analyze the behavior of the proposed retrieval system, we present qualitative retrieval examples on the MSVD test set. For each text query, the top-5 retrieved videos are shown ranked by similarity score. These examples provide insight into the semantic alignment learned between text and video representations.

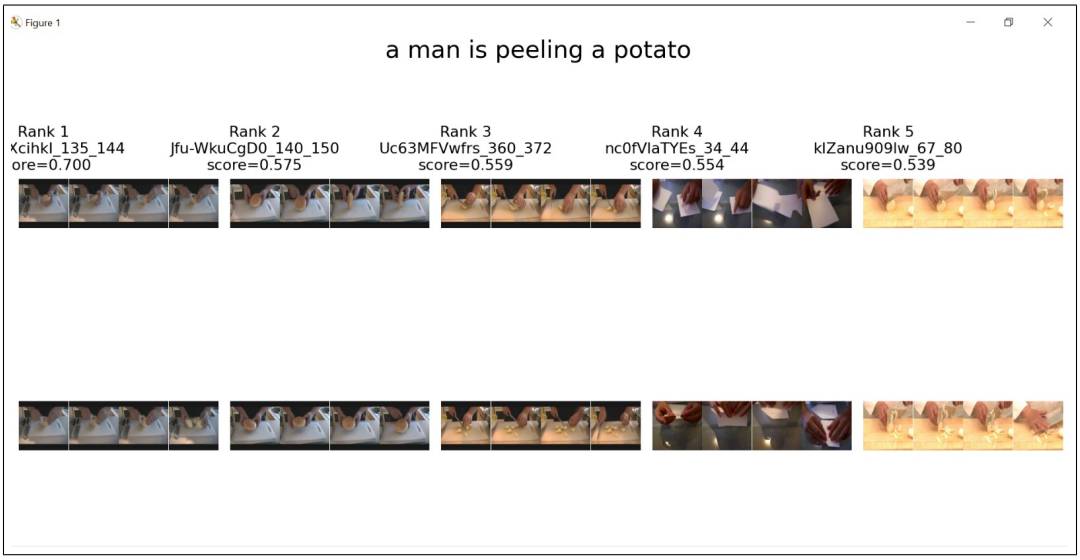

Figure 1: Top-5 retrieved videos for the query: "a man is peeling a potato".

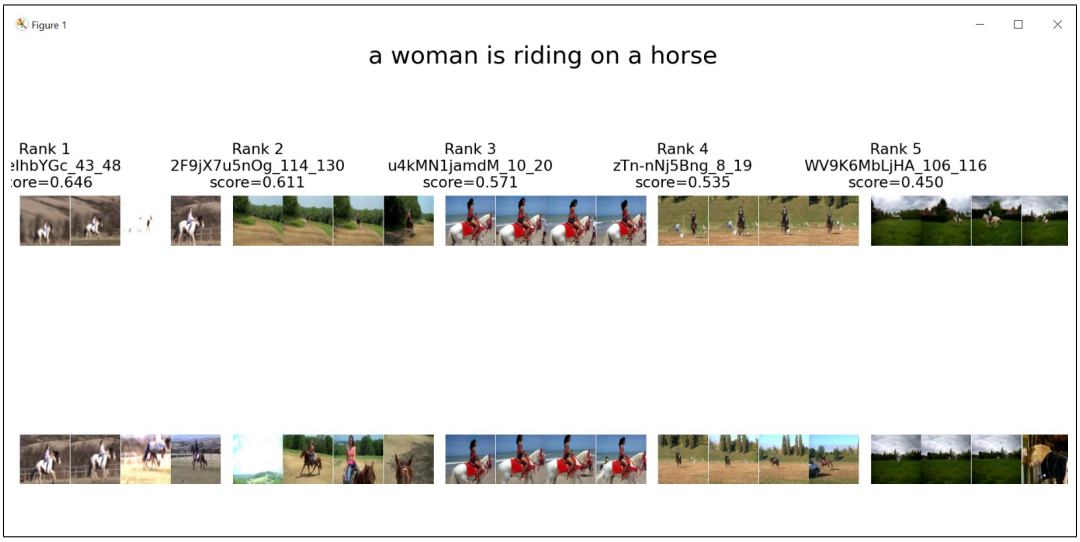

Figure 2: Top-5 retrieved videos for the query: "a woman is riding on a horse".

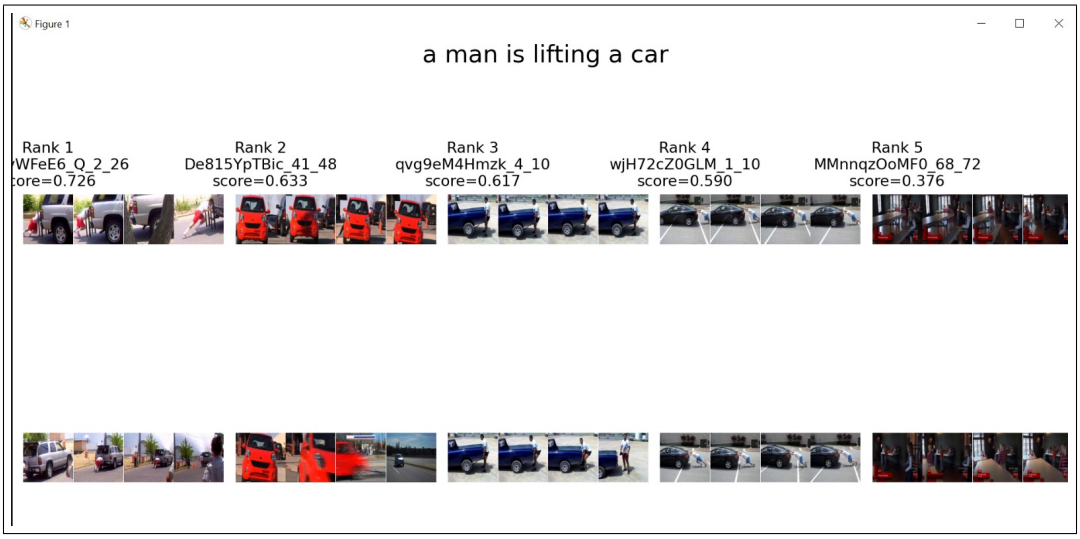

Figure 3: Top-5 retrieved videos for the query: "a man is lifting a car".

These examples show that the system is able to retrieve semantically relevant videos even when multiple visually similar candidates exist. In most cases, the correct video appears within the top-ranked results, demonstrating effective cross-modal alignment.

6 Comparative Analysis

6.1 Comparison with Fusion-Based Methods

Fusion-based methods combine video and text features earlier in the model to enable fine-grained interactions. This can improve alignment between specific words and specific visual cues. However, fusion models are generally slower at inference because the model must process each query-video pair jointly.

In contrast, our dual-encoder approach supports efficient retrieval by precomputing video embeddings once. It also keeps the system modular and easier to extend. The main drawback is that the encoders do not explicitly learn detailed word-to-frame interactions, which can affect fine-grained retrieval. However, trade-off is acceptable in retrieval scenarios where fast inference and scalability are critical.

6.2 Comparison with Large Pretrained Models (CLIP)

CLIP is trained on a very large number of image-text pairs, which typically gives strong generalization. However, CLIP is primarily designed for images, not videos.

Applying it to video retrieval usually requires extracting frames and aggregating them, similar to our approach, but it does not explicitly capture temporal dynamics.

For this project, building a custom dual-encoder system is more educational and allows direct control over design choices such as frame sampling, pooling strategy, and fine-tuning. CLIP remains a strong reference point for future improvements, especially if combined with better temporal modeling.

6.3 Comparison with Older Non-Deep-Learning Methods

Traditional approaches rely on handcrafted visual features (e.g., local descriptors) and simple text features (e.g., bag-of-words). These methods typically capture low-level patterns but struggle with semantic understanding. Our deep learning approach learns representations directly optimized for semantic matching, which explains the substantial improvement in retrieval quality compared to classical pipelines.

6.4 Quantitative Comparison on MSVD

Table 3 presents a quantitative comparison between the proposed method and representative prior approaches evaluated on the MSVD dataset. The comparison highlights the trade-off between retrieval accuracy and model complexity.

Table 3: Comparison with Prior MSVD Methods

Method	R@1	R@5	R@10	MedR
JSFusion (Yu et al., 2018)	10.2	31.2	43.2	13
CE (Liu et al., 2019)	19.8	49.0	63.8	6
HGR (Chen et al., 2020)	35.1	73.5	83.5	3
CLIP (Zero-shot)	∼30	∼60	∼70	∼5
Our Method	**20.3**	**48.9**	**61.6**	**6**

Although the proposed approach does not achieve state-of-the-art performance, the results are competitive given the lightweight architecture and the limited training data. This demonstrates that dual-encoder models combined with transfer learning can provide an effective and efficient baseline for text-to-video retrieval.

7 Conclusion and Future Work

7.1 Project Summary

We implemented a text-to-video retrieval system using a dual-encoder architecture with transfer learning and contrastive training. Videos are represented by sampling frames, extracting CNN features, and temporally pooling them. Text is represented using a pretrained Transformer. Both modalities are projected into a shared 256-dimensional space where similarity-based retrieval is performed. On MSVD, the system achieves reasonable retrieval performance given its lightweight design, with high Recall@5 indicating that correct videos are usually ranked near the top.

7.2 Strengths of the System

- **Practical efficiency:** fast retrieval due to precomputed video embeddings.
- **Effective transfer learning:** pretrained encoders reduce training cost and improve accuracy.
- **Strong empirical performance:** good recall values for short video clips.
- **Interpretability and modularity:** clear separation between video and text processing.

7.3 Limitations of the System

- **Simplistic temporal modeling:** average pooling discards temporal ordering information.
- **Short-video focus:** MSVD contains short clips; longer videos may require different strategies.
- **No cross-modal attention:** the model does not explicitly align specific words with specific frames.
- **Dataset scope:** generalization beyond MSVD is not guaranteed without additional evaluation.

7.4 Future Directions

Several improvements can extend this system:

- **Temporal Transformer for videos:** model relationships between sampled frames rather than averaging.
- **Adaptive sampling:** select frames based on content change instead of uniform sampling.
- **Cross-modal attention:** enable fine-grained alignment between words and visual regions/frames.
- **Larger datasets:** evaluate and train on larger video-text benchmarks for better robustness.
- **Efficient indexing:** use approximate nearest neighbor search for scaling to very large databases.

7.5 Concluding Remarks

We implemented a text-to-video retrieval system based on a dual-encoder architecture with transfer learning and contrastive learning. Video representations are obtained by uniformly sampling frames, extracting visual features using a CNN backbone, and applying temporal average pooling. Text queries are encoded using a pretrained Transformer model. Both modalities are projected into a shared 256-dimensional embedding space, where similarity-based retrieval is performed using cosine similarity. On the MSVD benchmark, the system demonstrates reasonable retrieval performance given its lightweight design, with Recall@5 showing that the correct video is frequently ranked among the top retrieved results, even when not placed first.

References

- MSVD dataset: `https://www.cs.utexas.edu/users/ml/clamp/videoDescription/`
- ResNet: K. He et al., "Deep Residual Learning for Image Recognition," 2016.
- BERT/Transformers: A. Vaswani et al., "Attention is All You Need," 2017.
- CLIP: A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," 2021.